

HRMS Platform

Complete Project Blueprint, Flow and Technical Stack

A production-grade, self-hosted Human Resource Management System with role-based access, geofenced attendance, leave, payroll and talent modules, built on Next.js and MongoDB and deployed in your own data center.

Document purpose	Single source of truth for developers and AI assistants building the platform
Reference products	Keka (experience and UX), greytHR (compliance depth and policy engine)
Frontend / Backend	Next.js (App Router) + TypeScript, shared service layer
Database	MongoDB replica set, self-hosted in your data center
Version / date	v1.0 - October 2026 (working name: OrgHub HRMS)

How to use this document with an AI assistant. Paste or attach the whole PDF at the start of a session, then ask for one module or one phase at a time (for example: 'Implement section 10, geofenced attendance, backend first'). Section numbers are stable, so you can reference them. Statutory (tax, PF, ESI) rules and legal text in this document are planning guidance only; confirm with a CA and labour-law professional before go-live.

Contents

1. Vision, Goals and Scope	3
2. Roles, Permissions and Access Control	4
3. Module Catalog	6
4. System Architecture	7
5. Technology Stack	9
6. Repository and Code Structure	11
7. MongoDB Data Model	12
8. Authentication and Security	15
9. Geofenced Attendance (Core Feature)	16
10. Leave and Workflow Engine	18
11. Payroll Engine	19
12. End-to-End Business Flows	20
13. API Design	21
14. UI/UX and Interactivity	22
15. Real-time, Notifications and Background Jobs	24
16. Data Center Deployment (Self-Hosted)	25
17. Quality, Testing and DevOps	27
18. Compliance (India) and Data Governance	28
19. Delivery Roadmap	29
20. Advanced Features Backlog	30
21. Risks and Rules for AI Assistants	31
22. Glossary	32

1. Vision, Goals and Scope

The platform is an **HRMS**: one system in which an organization manages the full employee lifecycle - hiring, onboarding, daily attendance, leave, payroll, expenses, performance and exit - with every user seeing only what their role allows.

1.1 Product goals

- **Replace spreadsheets and ad-hoc tools** for HR and accounts in your own organization first.
- **Keka-like experience**: fast, visual, mobile-first, one-click common actions, role-based dashboards.
- **greytHR-like rigor**: configurable policies, compliance-ready payroll, detailed reports, full audit trail.
- **Self-hosted**: all data, files and backups stay in your data center; you control keys and access.
- **Multi-tenant ready**: every record carries a companyId so the product can serve more companies later.
- **Advanced and interactive**: real-time updates, command palette, drag-and-drop, live calendars, maps, analytics.

1.2 Scale assumptions (design targets)

Dimension	Initial	Design headroom
Employees	50 - 500	5,000+ without re-architecture
Concurrent users	50 - 200	1,000+ (morning clock-in spike)
Punch events per day	1,000 - 5,000	50,000+
Availability	99.5%	99.9% with replica and failover
Data retention	8 years for payroll/HR records	Configurable per policy and law

1.3 Personas

Persona	Main needs
Super Admin	Configure company, users, roles, policies, integrations; view audit logs
HR Manager	Employee records, onboarding and exit, leave and attendance policies, corrections, announcements
Accountant / Finance	Salary structures, run payroll, payslips, reimbursements, statutory reports, bank files
Manager / Team Lead	Approve leave, attendance and expenses; see team availability; run reviews
Employee	Clock in/out, apply leave, view payslips, raise requests, submit expenses
Recruiter	Job openings, candidate pipeline, interviews, offers
Auditor	Read-only reports and audit logs

1.4 Non-goals for version 1

- No microservices (start as a modular monolith).
- No continuous background location tracking (only at punch time, unless a field-staff policy says otherwise).
- No full learning-management or social features until the core is stable.

2. Roles, Permissions and Access Control

Access is controlled with **RBAC plus scope**. A user has one or more roles; a role is a list of permissions; each permission has a scope that limits which records it applies to. Authorization is **always enforced on the server** (in the service layer), never only by hiding buttons in the UI.

2.1 Default role matrix

Legend: **F** = full (create, edit, delete) **W** = write within scope **A** = approve **R** = read **S** = self only **T** = own team **-** = no access

Module	Super Admin	HR	Accountant	Manager	Employee	Recruiter	Auditor
Company and org setup	F	W	-	-	-	-	R
Users, roles, permissions	F	-	-	-	-	-	R
Employee profile	F	F	R (limited)	R (T)	S + edit requests	-	R
Salary data	F	R	F	-	S	-	R
Attendance	F	F	R	A (T)	S	-	R
Leave	F	F	R	A (T)	S	-	R
Payroll run	F	R	F	-	S (payslip)	-	R
Expenses	F	R	A + pay	A (T)	S	-	R
Recruitment	F	F	-	R (interviewer)	-	F	R
Onboarding / exit	F	F	W (F and F)	W (T tasks)	S tasks	-	R
Performance	F	F	-	W (T)	S	-	R
Reports and analytics	F	F	F (finance)	R (T)	-	R (hiring)	R
Audit logs	F	R (own actions)	-	-	-	-	R
Announcements / helpdesk	F	F	R	R	R + raise ticket	-	R

2.2 Permission model

- Permission key format: **module.resource.action** - for example `attendance.punch.create`, `leave.request.approve`, `payroll.run.execute`, `employee.salary.view`.
- Scope values: **self**, **team** (direct and indirect reports), **department**, **location**, **company**.
- Roles are data (stored in MongoDB), so admins can create custom roles without code changes. System roles are seeded and locked.
- Effective permissions = union of permissions of all roles of the user, with the widest scope winning per permission key.
- Field-level rules: salary, bank, PAN/Aadhaar fields are returned only if the caller has the specific field permission.
- Sensitive actions require re-authentication or MFA step-up (viewing salary, exporting data, running payroll, changing roles).

2.3 Authorization check (reference logic)

```
// packages/core/auth/can.ts
export async function can(ctx: Ctx, perm: string, target?: { ownerId?: string; deptId?: string; locId?: string }) {
  const grant = ctx.permissions.get(perm); // { scope: 'self'|'team'|'department'|'location'|'company' }
  if (!grant) return false;
  switch (grant.scope) {
    case 'company': return true;
    case 'department': return ctx.deptIds.includes(target?.deptId!);
    case 'location': return ctx.locIds.includes(target?.locId!);
    case 'team': return await isInReportingLine(ctx.employeeId, target?.ownerId!);
  }
}
```

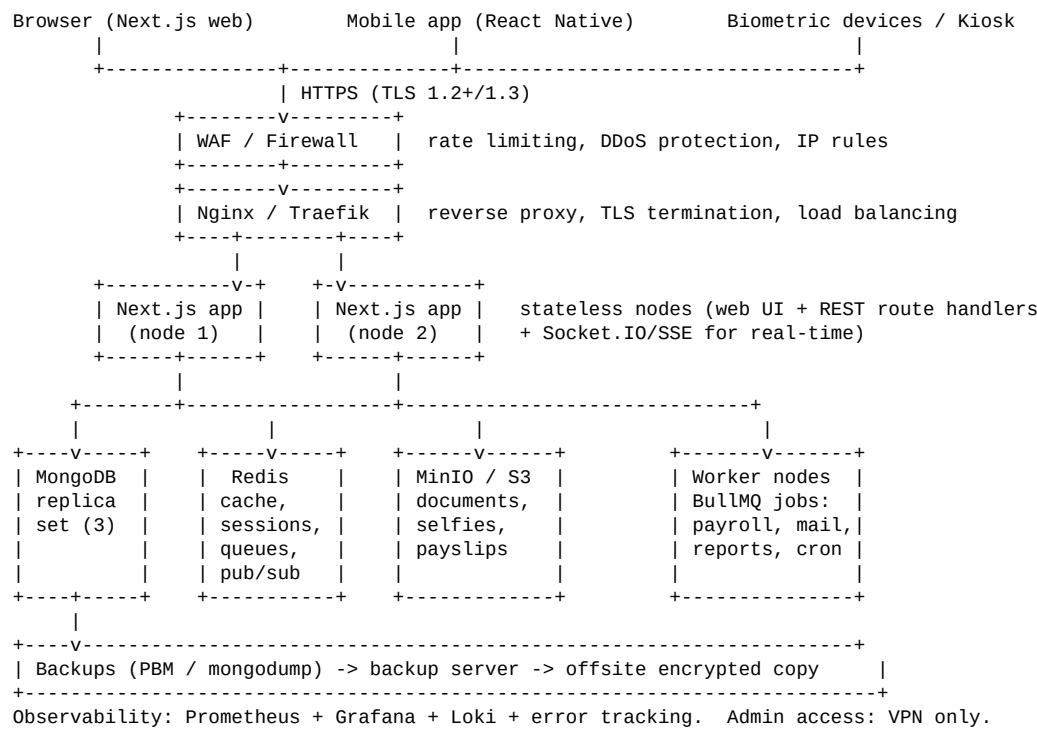
```
    case 'self':      return target?.ownerId === ctx.employeeId;
  }
}
// Every service method starts with: await assert(can(ctx, 'leave.request.approve', { ownerId }))
// Every query is ALSO filtered by { companyId: ctx.companyId } through a tenant-scoped repository.
```

3. Module Catalog

#	Module	Key capabilities	Phase
1	Authentication and users	Login, MFA (TOTP), password policy, SSO (optional), sessions and devices, account lockout, invite flow	1
2	Organization setup	Company, legal entities, departments, designations, grades, locations, cost centers, reporting hierarchy, custom fields	1
3	Core HR (employee master)	Profile, documents vault, bank details, emergency contacts, job and salary history (effective-dated), profile change requests with approval	1
4	Audit and compliance log	Immutable record of who did what, when, from where; salary-view and export logging	1
5	Attendance and geofencing	Geofenced punch, selfie, shifts and rosters, regularization, overtime, late rules, biometric import, timesheets	2
6	Leave management	Leave types, accrual, carry-forward, encashment, sandwich rule, half-day and hourly leave, team calendar, ledger	2
7	Workflow engine	Configurable multi-level approvals, escalation, delegation, reminders, reusable across modules	2
8	Holidays and calendar	Holiday lists by location, weekly offs, company events, shared calendar view	2
9	Payroll	Salary templates, components, formulas, LOP, arrears, bonuses, payslips, F and F, bank advice, statutory returns	3
10	Expenses and reimbursements	Claims with bills, policy limits, approvals, payout via payroll or bank file	3
11	Tax and declarations	Investment declarations, proof upload, TDS projection, Form 16 data	3
12	Recruitment (ATS)	Openings, careers page, candidate kanban, interviews, scorecards, offer letters	5
13	Onboarding and offboarding	Checklists, e-sign documents, asset handover, exit interview, F and F	5
14	Performance	Goals/OKRs, review cycles, 360 feedback, 1:1 notes, ratings calibration	5
15	Assets	Laptop/phone inventory, assignment, return, maintenance	5
16	Helpdesk and letters	Tickets with SLA, letter templates (offer, experience, salary certificate)	5
17	Engagement	Announcements, polls, kudos, birthdays and anniversaries, org chart, directory	5
18	Reports and analytics	Headcount, attrition, attendance trends, payroll cost, scheduled and exportable reports, dashboards	2-5
19	Notifications	In-app, email, push, SMS/WhatsApp (optional), templates, user preferences	1-2
20	Admin console	Policies, templates, custom roles, feature flags, integrations, data import/export	1-6

4. System Architecture

Start as a **modular monolith**: one repository, one deployable web application, one worker process, with strict module boundaries. This is far simpler to build, test and operate in your own data center than microservices, and modules can be split out later if scale requires.



4.1 Layers and responsibilities

Layer	Responsibility	Rules
UI (Next.js Server and Client Components)	Rendering, forms, interactivity	No business rules; call services or REST API; never trust client permission state
API (Route Handlers under /api/v1)	HTTP interface for web and mobile	Validate input with Zod, authenticate, call the service layer, return typed JSON
Service layer (packages/core)	All business logic and authorization	Single place for rules; used by API, server actions and workers
Repository / models (Mongoose)	Data access with tenant scoping	Every query adds companyId; no direct model use outside repositories
Worker	Long or scheduled work	Idempotent jobs, retries with backoff, dead-letter queue
Infrastructure	Mongo, Redis, MinIO, proxy	Private network, least privilege, encrypted

4.2 Next.js-specific guidance

- Use the **App Router** with Server Components for data-heavy pages (lists, reports) and Client Components for interactive widgets (calendars, maps, kanban).
- Expose a **versioned REST API** (/api/v1/...) because the mobile app and biometric integrations need it; web pages use the same service layer.
- **Never rely on Next.js middleware alone** for authorization. A 2025 middleware-bypass vulnerability showed why: re-check session and permissions inside every route handler, server action and service call. Keep Next.js patched.
- Run Next.js as a **self-hosted Node server** (output: standalone) in Docker; no serverless assumptions. Long jobs (payroll, report generation) go to the worker, not to request handlers.
- Server Actions are fine for simple forms, but they are public endpoints: validate and authorize exactly as you would an API route.

- Real-time: Socket.IO or SSE served from a custom Node process or a small sidecar service, fed by Redis pub/sub so it works across multiple app nodes.
- Use `instrumentation.ts` for tracing and logging setup; use Pino for structured JSON logs with a request ID.

4.3 Module boundaries (modular monolith)

Each module owns its models, services, validation schemas, policies and routes. Modules talk to each other only through exported service functions or domain events (an in-process event bus backed by BullMQ for async work), never by importing another module's models.

Events (examples): `employee.created` `employee.exited` `leave.approved` `attendance.day_closed`
 `payroll.run_locked` `payslip.published` `workflow.step_completed`
Consumers: `notifications`, `audit`, `analytics counters`, `leave-balance updater`, `payroll input collector`

5. Technology Stack

Layer	Choice	Why / notes
Language	TypeScript (strict) everywhere	Shared types between web, API, worker and mobile
Web framework	Next.js (App Router), React	SSR, routing, API routes, large ecosystem; self-hosted standalone build
Styling / UI kit	Tailwind CSS + shadcn/ui (Radix)	Accessible primitives, fully customizable design system
Data fetching	TanStack Query	Caching, optimistic updates, background refetch, retry
Tables	TanStack Table (+ virtualization)	Sorting, filtering, column chooser, server-side pagination
Forms and validation	React Hook Form + Zod	One Zod schema shared by client and server
UI state	Zustand (small global state)	Keep server state in TanStack Query, not in stores
Charts	Recharts or Apache ECharts	Dashboards with drill-down
Calendar	FullCalendar	Leave, attendance and roster views
Maps / geofence UI	Leaflet + OpenStreetMap (react-leaflet), Leaflet-Geoman for drawing	Free and self-hostable; optional self-hosted tile server
Drag and drop	dnd-kit	Kanban (recruitment), roster builder
Command palette / toasts	cmdk, Sonner	Ctrl+K global search, instant feedback
Animation	Framer Motion (sparingly)	Smooth transitions without hurting performance
Internationalization	next-intl	Multi-language and date/number formats
Backend logic	Service layer in TypeScript, exposed via Next.js route handlers	One codebase, shared with workers
ODM	Mongoose	Mature MongoDB ODM with schemas, hooks, transactions (Prisma's MongoDB support is more limited)
Database	MongoDB (current stable Community, replica set)	Flexible documents, 2dsphere geospatial index, multi-document transactions on replica sets
Cache / queue / pub-sub	Redis + BullMQ	Sessions, rate limiting, jobs, real-time fan-out
File storage	MinIO (S3 API) or Ceph / SeaweedFS	Self-hosted object storage; verify MinIO's current license and distribution terms before adopting
Auth	Auth.js (NextAuth) with credentials + TOTP, or Keycloak for SSO	Argon2id hashing, short-lived access token, rotating refresh token for mobile
PDF generation	Puppeteer (HTML to PDF) or React-PDF	Payslips, letters, reports
Email	Self-hosted SMTP relay (Postfix) or provider SMTP, React Email templates	Transactional mail
Push notifications	FCM / APNs	Third-party service even when self-hosted
Mobile app	React Native (Expo dev build)	Native location, camera, secure storage, Play Integrity; shares TypeScript types
Testing	Vitest, Testing Library, Playwright, k6	Unit, component, end-to-end, load
CI/CD	GitLab CE or Gitea + runners	Self-hosted pipeline; Trivy and npm audit for scanning
Containers / orchestration	Docker; Docker Compose first, k3s/Kubernetes later	Start simple; scale when needed
Virtualization	Proxmox or VMware	Isolate app, database, storage, backup
Secrets	HashiCorp Vault (or Vault-compatible OpenBao) / Docker secrets	Never in code or images
Observability	Prometheus, Grafana, Loki, Alertmanager, GlitchTip/Sentry (self-hosted)	Metrics, logs, alerts, error tracking
Config management	Ansible (+ Terraform if using virtualization APIs)	Repeatable server builds

Important MongoDB notes for self-hosting. (1) MongoDB Community does not include native encryption at rest; use full-disk encryption (LUKS) on database and backup volumes, plus application-level field encryption for the most

sensitive fields. (2) Transactions require a replica set, so never run a single standalone instance in production. (3) Review MongoDB's current license (SSPL for Community) with your legal team; internal use is normally fine, but selling the platform as a hosted service later may need review. (4) Use Percona Backup for MongoDB (open source) for consistent backups and point-in-time recovery.

6. Repository and Code Structure

```
hrms/
  apps/
    web/                Next.js app (UI + /api/v1 route handlers)
    app/
      (auth)/login, mfa, forgot-password
      (app)/dashboard, employees, attendance, leave, payroll, expenses,
        recruitment, performance, assets, helpdesk, reports, settings
      api/v1/...         REST route handlers (thin: auth + validate + call service)
      components/        ui/ (shadcn), charts/, calendar/, map/, tables/, forms/
      lib/               api-client, query-keys, hooks, permissions (UI hints only)
    worker/             BullMQ processors + cron schedulers
    mobile/             React Native app (attendance, leave, payslips)
    realtime/           Socket.IO / SSE gateway (optional separate process)
  packages/
    core/               business logic, one folder per module
      auth/ org/ employee/ attendance/ leave/ workflow/ payroll/ expense/
      recruitment/ performance/ assets/ helpdesk/ notifications/ audit/ reports/
      each module: model.ts repository.ts service.ts schema.ts (Zod) policy.ts events.ts
    db/                 Mongo connection, base schema plugin, migrations, seeds, index definitions
    shared/             types, constants, permission keys, error classes, date/money utils
    config/             env validation (Zod), feature flags
    ui/                 shared design tokens (optional)
  infra/               docker-compose, nginx, ansible, mongo keyfile templates, backup scripts
  docs/                ADRs, runbooks, API docs (OpenAPI), this blueprint
```

6.1 Coding conventions

- Every service function signature starts with ctx (companyId, userId, employeeId, roles, permissions, requestId).
- Errors are typed (NotFound, Forbidden, ValidationError, Conflict) and mapped to HTTP codes in one place.
- No business logic in React components or route handlers.
- All writes that change money, balances or approvals use a transaction and record an audit entry.
- Dates are stored in UTC; attendance also stores a local business date string (YYYY-MM-DD) and the timezone.
- Money is stored as integer minor units (paise) or Decimal128, never floating point.

7. MongoDB Data Model

7.1 Modeling rules

- **Base fields on every document:** companyId, createdAt, updatedAt, createdBy, updatedBy, isDeleted (soft delete), version (optimistic locking).
- **Embed** data that is read together and bounded (address, emergency contacts, salary components of one payslip snapshot). **Reference** data that is shared or unbounded (employee to department, punches to employee).
- **Never hard-delete** HR, attendance or payroll data. Use soft delete plus retention jobs.
- **Effective dating:** salary, designation, manager and location changes are new records with effectiveFrom; never overwrite history.
- **Immutable snapshots:** finalized payslips and approved attendance days store the full calculated values and the rule version used.
- **Idempotency keys** on punch, payroll run and payment-style operations to make retries safe.
- **Schema validation:** Mongoose schemas plus MongoDB JSON-Schema validators on critical collections.
- **Indexes:** every query path has an index; follow equality-sort-range order; always lead with companyId; review slow queries with the profiler monthly.
- **Transactions** (replica set): leave approval plus balance update, payroll finalization, expense payout, user and role changes.
- **Sensitive fields** (bank account, PAN, Aadhaar, salary): application-level encryption with keys held in Vault; mask in UI by default.

7.2 Collections

Collection	Key fields	Indexes and notes
companies	name, legalName, timezone, currency, fiscalYearStart, settings, statutory ids	unique domain; one doc per tenant
users	email, passwordHash, mfa{enabled,secret}, status, roleIds[], employeeId, lastLogin, failedAttempts	unique (companyId,email)
roles	name, isSystem, permissions[{key,scope}]	unique (companyId,name)
employees	empCode, personal{}, contact{}, job{deptId,designationId,gradeId,managerId,locationId,type,doj,status}, bank{enc}, ids{enc}, customFields{}	unique (companyId,empCode); (companyId,managerId); (companyId,deptId,status); text index for search
employee_history	employeeId, field, oldValue, newValue, effectiveFrom, changedBy	(companyId,employeeId,effectiveFrom)
employee_documents	employeeId, type, fileKey, status, expiry, verifiedBy	(companyId,employeeId,type)
departments / designations / grades	name, code, parentId, headId	unique (companyId,code)
work_locations	name, address, center (GeoJSON Point), radiusMeters, polygon (GeoJSON, optional), wifiBssids[], qrSecret, timezone, active	2dsphere on center and on polygon; (companyId,active)
employee_locations	employeeId, locationId, validFrom, validTo, mode (fixed flexible remote field)	(companyId,employeeId)
attendance_policies	name, geofenceMode (strict soft off), selfieRequired, maxAccuracyM, sources[], graceMin, halfDayHours, fullDayHours, latePenaltyRules[], autoPunchOut	versioned; assigned per shift or employee group
shifts / rosters	name, start, end, breakMin, crossesMidnight, weeklyOffs[]; roster: employeeId, date, shiftId	(companyId,employeeId,date)
attendance_punches	employeeId, type(IN OUT), serverTs, deviceTs, loc (GeoJSON Point), accuracyM, matchedLocationId, distanceM, source, deviceId, ip, selfieKey, flags[], status, idempotencyKey	(companyId,employeeId,serverTs desc); 2dsphere on loc; unique idempotencyKey; immutable (append-only)

Collection	Key fields	Indexes and notes
attendance_days	employeeId, date, shiftId, firstIn, lastOut, workedMin, status(P A HD L H WO OD WFH), lateMin, overtimeMin, punchIds[], locked, ruleVersion	unique (companyId,employeeId,date)
regularization_requests	employeeId, date, requestedIn/Out, reason, workflowId, status	(companyId,status)
holidays	locationIds[], date, name, type	(companyId,date)
leave_types / leave_policies	code, paid, accrual{freq,amount}, carryForward, maxBalance, encashable, sandwichRule, halfDay, applicableTo{}	versioned
leave_balances	employeeId, leaveTypeId, year, opening, accrued, used, adjusted, closing	unique (companyId,employeeId,leaveTypeId,year)
leave_ledger	employeeId, leaveTypeId, delta, reason, refId, ts	append-only; balance = sum(ledger)
leave_requests	employeeId, leaveTypeId, from, to, dayParts[], days, reason, status, workflowId	(companyId,employeeId,from); (companyId,status)
workflow_definitions	key (leave, expense, regularization...), steps[{approverRule, sla, escalateTo, onTimeout}], conditions	unique (companyId,key)
workflow_requests	definitionKey, subjectRef, currentStep, steps[{approverId,status,actedAt,comment}], status	(companyId,approverId,status)
salary_components	code, name, type(earning deduction employer), taxable, formula, statutory flags	versioned
salary_structures	name, components[{code, formula fixed}]	versioned
employee_salary	employeeId, structureId, ctc, overrides{}, effectiveFrom	(companyId,employeeId,effectiveFrom desc); field-encrypted
payroll_runs	period, status(draft calculated review approved locked paid), counts, totals, ruleVersion, lockedBy	unique (companyId,period)
payslips	runId, employeeId, snapshot{earnings,deductions,employer,tax,net, attendanceSummary}, pdfKey, published	unique (runId,employeeId); immutable after lock
expense_claims / categories	employeeId, items[{date,category,amount,billKey}], status, workflowId, payoutRef	(companyId,employeeId,status)
tax_declarations	employeeId, fy, regime, items[], proofs[], status	unique (companyId,employeeId,fy)
job_openings / candidates / interviews	stage, scorecards, offer{}, source	(companyId,openingId,stage)
onboarding_templates / tasks	checklist items, owner role, due offsets	
goals / review_cycles / reviews	cycle, ratings, feedback, calibration	(companyId,cycleId,employeeId)
assets / asset_assignments	tag, type, serial, status, assignee, dates	unique (companyId>tag)
tickets / announcements / notifications	category, SLA, audience, readBy[]	TTL on old notifications
audit_logs	actorId, action, entity, entityId, before, after, ip, userAgent, ts, requestId	append-only; (companyId,entity,entityId,ts); restricted delete
idempotency_keys / sessions / devices	key, response hash, ttl; refresh token family; registered deviceId per employee	TTL indexes

7.3 Example documents

```
// work_locations
{ _id, companyId, name: "Mumbai HQ",
  center: { type: "Point", coordinates: [72.8777, 19.0760] }, // [longitude, latitude]
  radiusMeters: 150, polygon: null, wifiBssids: ["aa:bb:cc:dd:ee:ff"], timezone: "Asia/Kolkata", active: true }

// attendance_punches (append-only)
{ _id, companyId, employeeId, type: "IN", serverTs: ISODate(), deviceTs: ISODate(),
  loc: { type: "Point", coordinates: [72.8779, 19.0762] }, accuracyM: 18,
  matchedLocationId, distanceM: 31, source: "mobile", deviceId, ip, selfieKey: "selfies/2026/10/..",
  flags: [], status: "valid", idempotencyKey: "emp123-2026-10-01T03:15:00Z-IN" }
```

```
// payslips (immutable snapshot)
{ _id, companyId, runId, employeeId, period: "2026-10", ruleVersion: "IN-2026.2",
  snapshot: { earnings: [{code:"BASIC",amt:NumberDecimal("40000.00")}, ...], deductions: [...],
    employer: [...], tds: ..., net: NumberDecimal("...") , paidDays: 30, lopDays: 0 }, published: false }
```

7.4 Required indexes (create at startup via migrations)

```
db.work_locations.createIndex({ center: "2dsphere" })
db.work_locations.createIndex({ polygon: "2dsphere" }, { sparse: true })
db.attendance_punches.createIndex({ companyId: 1, employeeId: 1, serverTs: -1 })
db.attendance_punches.createIndex({ idempotencyKey: 1 }, { unique: true })
db.attendance_days.createIndex({ companyId: 1, employeeId: 1, date: 1 }, { unique: true })
db.employees.createIndex({ companyId: 1, empCode: 1 }, { unique: true })
db.leave_requests.createIndex({ companyId: 1, employeeId: 1, from: -1 })
db.workflow_requests.createIndex({ companyId: 1, "steps.approverId": 1, status: 1 })
db.audit_logs.createIndex({ companyId: 1, entity: 1, entityId: 1, ts: -1 })
db.notifications.createIndex({ createdAt: 1 }, { expireAfterSeconds: 7776000 }) // 90 days
```

8. Authentication and Security

8.1 Authentication flows

- 1 **Invite and activation:** HR creates an employee; system emails a one-time invite link; user sets password (Argon2id) and enrolls MFA.
- 2 **Login:** email and password, then TOTP code (or SSO). Failed attempts increment a counter; lock after 5 failures with cool-down.
- 3 **Sessions (web):** HTTP-only, Secure, SameSite cookies; short session with idle timeout; list and revoke active sessions.
- 4 **Tokens (mobile):** 15-minute access token + rotating refresh token stored in the device secure store; reuse of an old refresh token revokes the whole family.
- 5 **Device binding:** each employee registers one mobile device for attendance; replacing it needs HR approval.
- 6 **Step-up auth:** re-prompt for MFA before salary view, payroll run, data export, role change.
- 7 **Password reset:** single-use expiring token; all sessions revoked after reset.

8.2 Security controls checklist

Area	Controls
Transport	TLS everywhere including internal Mongo/Redis traffic; HSTS; modern ciphers
Application	Zod validation on every input; output encoding; CSRF protection; strict Content-Security-Policy; secure headers; file-type and size checks on uploads with antivirus scan (ClamAV)
Authorization	Server-side RBAC with scope on every service call; tenant filter on every query; no IDOR (always verify ownership)
Data	Disk encryption (LUKS) for DB, files and backups; field-level encryption for bank/ID/salary; masking in UI; export watermarking and logging
Secrets	Vault or secret store; rotate DB passwords, JWT keys and encryption keys on a schedule
Database	Authentication on, keyfile or x.509 between members, least-privilege DB users (app, worker, backup, read-only analytics), bind to private IP only, no public exposure
Network	Segmentation: DMZ (proxy), app tier, data tier; firewall default-deny; VPN for admin; bastion host; fail2ban
Abuse protection	Rate limiting by IP and user (Redis); bot protection on login; WAF rules
Monitoring	Alerts on failed-login spikes, mass exports, impossible-travel punches, permission changes
Software supply chain	Lockfiles, dependency scanning, image scanning, signed builds, regular patching
Assurance	Annual third-party penetration test; quarterly access review; incident response plan

9. Geofenced Attendance (Core Feature)

9.1 Concept

Attendance is recorded as **immutable punch events**. A daily job (and on-demand recalculation) turns punches into an **attendance day** using shift and policy rules. Location is checked at the moment of punch against the employee's assigned work locations, and the decision is made **on the server**, never by the client.

9.2 Setup by HR / Admin

- 1 Create **work locations** on a map: drop a pin (center) and set a radius (default 150 m), or draw a polygon for a campus.
- 2 Optionally add **Wi-Fi BSSIDs** or a rotating **QR code** as secondary proof for places with weak GPS (basements, towers).
- 3 Create an **attendance policy**: strict or soft geofence, selfie on/off, max GPS accuracy, allowed sources, grace time, half and full day hours.
- 4 **Assign** employees or groups to locations and policies; mark remote, field or work-from-home modes as needed.
- 5 Approved **OD (on duty)** or WFH requests automatically relax geofence rules for those dates.

9.3 Punch flow (end to end)

```
Employee taps CLOCK IN
|
v
Mobile app: get GPS fix (high accuracy) + accuracy radius + isMockLocation flag + device id
            + Play Integrity / device attestation token + (selfie if policy requires)
|
v POST /api/v1/attendance/punch (HTTPS, Bearer token, Idempotency-Key header)
Server pipeline (each step can reject or flag):
1. Authenticate; load employee, policy, shift, assigned locations
2. Device check: deviceId matches registered device; attestation token valid
3. Mock-location / rooted-device flag -> reject or flag per policy
4. Accuracy check: accuracyM <= policy.maxAccuracyM (else ask user to retry outdoors)
5. Time check: |deviceTs - serverTs| within tolerance; server time is the source of truth
6. Geofence check: $geoNear against assigned locations (see 9.4)
7. Sequence check: no double IN; OUT requires open IN (or follow policy for multi-punch)
8. Velocity check: impossible travel vs last punch
9. Selfie liveness / face-match (optional, async score)
10. Decision: VALID | PENDING_APPROVAL (soft policy, outside fence) | REJECTED (reason code)
|
v
Save punch (append-only) -> update today's attendance_day -> publish real-time event
-> notify manager if pending -> respond with status, distance, matched location
```

9.4 Server-side geofence check (MongoDB)

```
// packages/core/attendance/geofence.ts
export async function matchLocation(companyId, locationIds, lng, lat) {
  const [hit] = await WorkLocation.aggregate([
    { $geoNear: {
      near: { type: 'Point', coordinates: [lng, lat] },
      distanceField: 'distanceM', spherical: true,
      query: { companyId, _id: { $in: locationIds }, active: true } },
    { $match: { $expr: { $lte: ['$distanceM', '$radiusMeters'] } } },
    { $limit: 1 },
  ]]);
  return hit ?? null; // { _id, name, distanceM, radiusMeters }
}

// Polygon locations:
WorkLocation.findOne({ _id: { $in: ids }, polygon: { $geoIntersects: { $geometry: { type: 'Point', coordinates: [lng, lat] } } } })
// Remember: GeoJSON order is [longitude, latitude].
```

9.5 Anti-spoofing and reliability matrix

Risk / problem	Mitigation
Fake GPS app (Android)	Detect mock provider flag, developer options, rooted device; Google Play Integrity attestation verified on the server

Risk / problem	Mitigation
Jailbroken iPhone	Jailbreak detection and App Attest; fewer spoofing options on iOS
Web punch spoofing (browser location is easy to fake)	Make the mobile app the primary channel; web punch allowed only as soft mode needing approval, or limited to office IP ranges
Buddy punching	Device binding (one device per employee) + selfie with liveness check + face match score
Phone clock tampering	Use server timestamp only; keep deviceTs for diagnostics
Teleporting (punch in two places)	Velocity check between consecutive punches; flag impossible travel
Poor indoor GPS (20-100 m error)	Capture accuracy; require it under the policy limit; realistic radius (100-200 m); Wi-Fi BSSID or QR fallback
Location permission denied	Block punch and show step-by-step help; log the attempt
No internet at the gate	Mobile queues the punch offline with device time and location, syncs later, marked 'offline punch' for review
Many people from one device or IP	Server heuristic flags and alerts HR
Forgotten punch-out	Auto punch-out at a configured time flagged for regularization; employee submits a request via workflow

9.6 Configurable policy options

- Strict (block outside fence) vs soft (accept with reason, requires manager approval) vs off (field/remote staff).
- Selfie: off / required / required only when outside fence.
- Allowed sources: mobile, web, kiosk, biometric device import.
- Grace period for late arrival; late-mark penalties (for example 3 late marks = half day).
- Minimum hours for full day and half day; overtime rules; comp-off generation.
- Multiple punches per day (break in/out) vs first-in/last-out computation.
- Auto punch-out time; maximum allowed shift length; punch window around shift start and end.

9.7 Attendance day calculation

- 1 Collect all valid and approved punches for the employee for the shift date (night shifts belong to the date they start).
- 2 Compute firstIn, lastOut, worked minutes, break minutes; apply grace, late and early-exit rules.
- 3 Apply approved leave, holidays, weekly offs, OD, WFH, comp-offs and regularizations.
- 4 Assign status: Present, Absent, Half Day, Leave, Holiday, Weekly Off, OD, WFH; compute LOP.
- 5 Store rule version used; lock the month at payroll cut-off; later changes require an unlock with audit trail.

9.8 Privacy and legal

- Collect location only at punch time. Continuous tracking only for explicitly defined field-staff policies.
- Show a consent screen and publish an attendance and location policy: what is collected, why, retention and who can view it.
- Restrict raw coordinates and selfies to HR and the employee; set retention (for example 90 days for selfies and raw location, longer for decisions).
- India's DPDP Act expects notice, consent and purpose limitation; have legal counsel review the policy.

9.9 Biometric and kiosk integration

Biometric devices (such as ZKTeco or eSSL) push logs to an ingestion endpoint or are polled by a connector. Imported logs are mapped to employees by device user id and stored in `attendance_punches` with `source = biometric`, so the same attendance-day engine processes all sources.

10. Leave and Workflow Engine

10.1 Leave rules to support

- Leave types: casual, sick, earned/privilege, maternity/paternity, comp-off, unpaid, custom.
- Accrual: monthly, quarterly, yearly, on joining, pro-rata for mid-year joiners; carry-forward limits; expiry; encashment.
- Half-day and hourly leave; sandwich rule (holidays/weekly offs between leave days); minimum notice; maximum consecutive days; document required.
- Balance preview before applying, with clash warning (team members already off).
- Balance is derived from an append-only **leave ledger**; approvals post ledger entries inside a transaction.

10.2 Leave request flow

- 1 Employee selects dates; UI shows remaining balance, holidays in range, team clash, and policy hints.
- 2 Server validates policy, balance, overlaps, notice period; creates `leave_request` and a `workflow_request`.
- 3 Workflow routes to the reporting manager (then HR for long leave, if configured); reminders and escalation on SLA breach.
- 4 On approval: transaction posts ledger entry, updates balance, marks attendance days as Leave, publishes event, notifies employee and calendar updates live.
- 5 Cancellation and reversal post compensating ledger entries (never delete).

10.3 Workflow engine (reused everywhere)

A generic engine handles any request that needs approval: leave, regularization, expenses, profile changes, asset requests, letters, offers. Definitions are data, editable by admins.

```
{ key: "leave", conditions: [{ if: "days > 5", steps: ["manager", "hr"] }, { else: ["manager"] }],
  steps: [{ name: "manager", approver: { type: "reportingManager" }, slaHours: 24,
    escalateTo: { type: "managersManager" }, onTimeout: "escalate" },
    { name: "hr", approver: { type: "role", role: "HR" }, slaHours: 48, onTimeout: "remind" }],
  delegation: true, autoApproveIf: null }
// Engine features: multi-level, parallel approvers (any/all), delegation when approver is on leave,
// escalation, reminders, comments, audit of each action, withdraw by requester.
```

11. Payroll Engine

11.1 Principles

- Payroll is the hardest and highest-risk module; build it after Core HR, attendance and leave are stable.
- Salary components and formulas are **configuration**, evaluated by a safe expression engine (never JavaScript eval).
- Statutory rules (PF, ESI, Professional Tax, TDS, LWF, gratuity, bonus) live in a **versioned rules layer** by state and financial year, so Finance Act changes need no redeploy.
- Finalized payslips are **immutable snapshots**; corrections are made as arrears or adjustments in a later run.
- Run **parallel payroll** against your current process for 2-3 cycles and reconcile before going live.
- Use Decimal128 or integer paise with explicit rounding rules.

11.2 Salary structure example

Component	Type	Formula / rule
Basic	Earning	40% of CTC / 12
HRA	Earning	50% of Basic (metro) or 40% (non-metro), configurable
Special allowance	Earning	CTC/12 minus all other earnings and employer contributions
Employee PF	Deduction	12% of PF wage (rules for wage ceiling in configuration)
Professional tax	Deduction	State slab table by monthly gross
TDS	Deduction	Projected annual tax by chosen regime, spread across remaining months
LOP	Deduction	(Gross / calendar or working days) x LOP days
Employer PF / ESI	Employer cost	Per statutory rules and ceilings

11.3 Payroll run state machine

```
DRAFT --(collect inputs)--> INPUTS_READY --(calculate)--> CALCULATED --(HR/Finance review, fix, recalc)--> REVIEW
--(approve, 2-person rule)--> APPROVED --(lock)--> LOCKED --(publish payslips, bank file)--> PAID
Unlock after LOCK needs Super Admin + reason + audit entry; PAID cannot be unlocked (use adjustments).
```

11.4 Run steps

- 1 **Cut-off**: attendance month locked; leave, LOP, new joiners, exits, salary revisions, bonuses, arrears, reimbursements and loans collected.
- 2 **Calculate** (worker job, per employee, idempotent, resumable): gross, deductions, employer cost, tax, net; store warnings (negative net, missing bank, zero days).
- 3 **Review**: variance report versus last month, exceptions list, drill-down from net pay to each rule and input.
- 4 **Approve**: Finance prepares, a second authorized person approves.
- 5 **Lock and publish**: generate payslip PDFs (stored in MinIO), email and in-app notify, create bank advice file and statutory reports.
- 6 **Post-run**: payment confirmation import, reversal handling, year-end Form 16 data and returns support.

12. End-to-End Business Flows

12.1 Employee onboarding

- 1 Recruiter or HR creates the employee (or converts an accepted candidate); system generates empCode and a role-based onboarding checklist.
- 2 Invite email; employee completes profile, uploads documents, e-signs policies and offer.
- 3 Tasks auto-assigned: IT (laptop, email), Admin (ID card), Finance (bank/salary), Manager (buddy, first-week plan).
- 4 HR verifies documents; salary structure assigned with effective date; employee activated; shift, location and leave policies assigned.
- 5 Day-one dashboard shows checklist progress; events (employee.created) trigger accounts and notifications.

12.2 Daily attendance

- 1 Employee opens the app, sees today's shift and the Clock In button with live location status (inside / outside / low accuracy).
- 2 Punch is validated per section 9; the UI confirms instantly and the manager's live 'who is in' board updates.
- 3 End of day: Clock Out; the nightly job closes days, flags exceptions (missing punch, late, outside fence).
- 4 HR sees an exceptions inbox; employees regularize via workflow; managers approve; days recompute until month lock.

12.3 Monthly cycle

- 1 Day 1-25: normal operations; reminders for pending approvals.
- 2 Cut-off day: attendance and leave lock; HR resolves exceptions.
- 3 Payroll run per section 11; Finance approves; payslips published; bank file and statutory reports generated.
- 4 Month-end reports: attendance summary, headcount, payroll cost by department, attrition.

12.4 Employee exit

- 1 Resignation request through workflow; notice period computed from policy; manager and HR approval.
- 2 Offboarding checklist: asset return, access removal, knowledge transfer, exit interview.
- 3 Full and final settlement: pending salary, leave encashment, recoveries, gratuity if applicable, notice-period adjustments.
- 4 Account moves to inactive, data retained per policy, experience and relieving letters generated from templates.

12.5 Expense claim

- 1 Employee creates a claim, uploads bills (OCR optional), policy limits are checked live.
- 2 Workflow approval (manager, then finance for large amounts); finance marks approved items for payout.
- 3 Payout via payroll component or bank file; claim closed with payout reference.

13. API Design

13.1 Conventions

- Versioned base path `/api/v1`; JSON; ISO-8601 UTC timestamps; money as strings or minor units.
- Authentication: session cookie (web) or Bearer token (mobile); every request carries X-Request-Id.
- Pagination: cursor or page+limit with total; filtering and sorting through query params; field selection where useful.
- Errors: `{ error: { code, message, details[], requestId } }` with correct HTTP status.
- Idempotency-Key header on punch, payroll, payout and other retry-sensitive POSTs.
- Rate limits per user and IP; OpenAPI spec generated from Zod schemas and published to developers.

13.2 Endpoint groups (reference)

Group	Endpoints
Auth	POST <code>/auth/login</code> , <code>/auth/mfa/verify</code> , <code>/auth/refresh</code> , <code>/auth/logout</code> , <code>/auth/forgot</code> , <code>/auth/reset</code> ; GET <code>/auth/sessions</code> ; DELETE <code>/auth/sessions/:id</code>
Users and roles	GET/POST <code>/users</code> ; PATCH <code>/users/:id</code> ; GET/POST <code>/roles</code> ; PUT <code>/roles/:id/permissions</code> ; GET <code>/me/permissions</code>
Organization	CRUD <code>/departments</code> , <code>/designations</code> , <code>/grades</code> , <code>/locations</code> (work locations), <code>/holidays</code> ; GET <code>/org/chart</code>
Employees	GET/POST <code>/employees</code> ; GET/PATCH <code>/employees/:id</code> ; GET <code>/employees/:id/history</code> , <code>/documents</code> ; POST <code>/employees/:id/change-requests</code> ; POST <code>/employees/import</code>
Attendance	POST <code>/attendance/punch</code> ; GET <code>/attendance/today</code> ; GET <code>/attendance/days?from&to</code> ; POST <code>/attendance/regularize</code> ; GET <code>/attendance/exceptions</code> ; GET <code>/attendance/live</code> (who is in); POST <code>/attendance/biometric/ingest</code> ; CRUD <code>/attendance/policies</code> , <code>/shifts</code> , <code>/rosters</code>
Leave	GET <code>/leave/balances</code> ; POST <code>/leave/preview</code> ; POST <code>/leave/requests</code> ; POST <code>/leave/requests/:id/cancel</code> ; GET <code>/leave/calendar</code> ; CRUD <code>/leave/types</code> , <code>/leave/policies</code>
Workflow	GET <code>/approvals/inbox</code> ; POST <code>/approvals/:id/approve</code> <code>reject</code> <code>delegate</code> <code>comment</code> ; CRUD <code>/workflow/definitions</code>
Payroll	CRUD <code>/payroll/components</code> , <code>/payroll/structures</code> ; PUT <code>/payroll/employees/:id/salary</code> ; POST <code>/payroll/runs</code> ; POST <code>/payroll/runs/:id/calculate</code> <code>approve</code> <code>lock</code> <code>publish</code> ; GET <code>/payroll/runs/:id/variance</code> ; GET <code>/me/payslips</code>
Expenses	POST <code>/expenses/claims</code> ; GET <code>/expenses/claims</code> ; POST <code>/expenses/claims/:id/submit</code> ; approvals via workflow
Recruitment	CRUD <code>/jobs</code> , <code>/candidates</code> ; PATCH <code>/candidates/:id/stage</code> ; POST <code>/interviews</code> ; POST <code>/offers</code>
Performance	CRUD <code>/goals</code> , <code>/review-cycles</code> ; POST <code>/reviews/:id/submit</code>
Reports	GET <code>/reports/:key</code> (headcount, attrition, attendance, payroll-cost); POST <code>/reports/:key/export</code> ; scheduled reports CRUD
Files	POST <code>/files/presign</code> ; GET <code>/files/:key</code> (authorized, short-lived signed URL)
Notifications	GET <code>/notifications</code> ; PATCH <code>/notifications/:id/read</code> ; PUT <code>/me/notification-preferences</code>
Admin and audit	GET <code>/audit-logs</code> ; GET/PUT <code>/settings</code> ; GET <code>/health</code> , <code>/ready</code> ; feature flags

14. UI/UX and Interactivity

14.1 Design principles

- 1 **Role-based home:** each role lands on a dashboard built for its daily tasks.
- 2 **One-click frequent actions:** Clock In, Apply Leave, Download Payslip, Approve - available from the home screen and the command palette.
- 3 **Instant feedback:** skeleton loaders, optimistic updates, toasts, inline validation; no full page reloads.
- 4 **Smart forms:** defaults, live balance, clash warnings, autosave drafts, multi-step wizards for complex tasks.
- 5 **Every list has four states:** loading, empty (with a call to action), error (with retry), populated.
- 6 **Mobile-first and responsive;** touch-friendly targets; installable PWA for non-attendance features.
- 7 **Accessible:** WCAG 2.1 AA, keyboard navigation, focus states, labels, color-contrast checked, never color-only meaning.
- 8 **Consistent design system:** tokens (color, type, spacing, radius, shadow), light and dark themes, Figma first then code.

14.2 Dashboards by role

Role	Widgets
Employee	Clock In/Out card with location status and timer, today's shift, leave balances, upcoming holidays, pending requests, latest payslip, announcements, birthdays
Manager	Who is in/out today (live), approvals inbox, team leave calendar, late and exception list, team goals
HR	Headcount and joiners/leavers, attendance exceptions, pending onboarding/exit tasks, leave trends, document expiry, helpdesk queue
Accountant	Payroll run status and checklist, variance alerts, reimbursements to pay, statutory due dates, cost by department
Super Admin	System health, user activity, security alerts, policy changes, audit highlights
Recruiter	Open positions, pipeline funnel, interviews today, offers pending

14.3 Sitemap (main navigation)

Home | My Space (profile, attendance, leave, payslips, documents, expenses, requests)
Team (approvals, team calendar, who is in) | People (directory, org chart, employee records)
Attendance (live board, exceptions, shifts and rosters, policies, geofence map) | Leave (policies, calendar, balances)
Payroll (runs, salary structures, statutory, payslips) | Expenses | Hiring | Performance | Assets | Helpdesk
Reports | Settings (company, roles and permissions, workflows, templates, integrations, audit logs)

14.4 Interactive features to implement

Feature	Behavior
Clock In widget	Requests location, shows a live map with the geofence circle and the user's dot, accuracy badge, one big button; success animation; offline queue indicator
Geofence editor (admin)	Draw circle or polygon on map, search address, adjust radius slider, preview employees covered, test a coordinate
Attendance calendar	Color-coded days (present, absent, late, leave, holiday, weekly off); click a day for punches, map and regularize action
Live 'who is in' board	Real-time updates through WebSocket/SSE; filters by team, location, status
Leave calendar	Team view with overlay of holidays and clashes; drag to select dates
Approvals inbox	Bulk approve, keyboard shortcuts, inline comments, swipe actions on mobile
Command palette (Ctrl+K)	Global search for people, pages, actions (apply leave, download payslip)
Org chart	Zoom, pan, collapse, search, click to open profile
Kanban boards	Drag-and-drop for candidates and onboarding tasks with optimistic updates
Roster builder	Drag employees onto shifts, conflict detection, publish and notify

Feature	Behavior
Data tables	Server-side pagination, column chooser, saved views, filters, bulk actions, export to Excel/CSV/PDF
Payroll wizard	Step-by-step: inputs, calculate (progress bar via real-time job events), review variances, approve, publish
Dashboards and charts	Filters, drill-down, date presets, export, scheduled email
Notifications center	Real-time bell, grouped, mark read, preferences per channel
Theming and preferences	Dark mode, density, language, saved filters
Guided onboarding / empty states	Product tours, checklists, sample data in demo mode

14.5 Performance targets

- Largest Contentful Paint under 2.5 s on 4G; interaction response under 200 ms; API p95 under 400 ms for common reads.
- Use streaming and Suspense, route-level code splitting, image optimization, virtualized long lists, and Redis caching for read-heavy lookups.
- Debounce search, prefetch likely next pages, and keep bundle size budgets in CI.

15. Real-time, Notifications and Background Jobs

15.1 Real-time events

- Redis pub/sub fan-out to Socket.IO/SSE gateway; rooms per company, team and user.
- Events: punch.recorded, approval.assigned, approval.updated, payroll.progress, notification.created, announcement.published.
- Clients subscribe after authentication; the server enforces room access with the same permission checks.

15.2 Background jobs (BullMQ)

Queue / job	Trigger	Notes
attendance.close_day	Nightly per timezone + on demand	Builds attendance_days; idempotent
attendance.auto_punch_out	Shift end + buffer	Flags for regularization
leave.accrual	Monthly / yearly cron	Posts ledger entries
workflow.sla_check	Every 15 min	Reminders and escalations
payroll.calculate	User action	Chunked, resumable, progress events
payslip.generate_pdf	After lock	Puppeteer pool; stores to MinIO
notification.send	Event driven	Email, push, SMS with retry and templates
report.generate / scheduled	Cron or user	Large exports built asynchronously then downloaded
biometric.sync	Interval	Ingest device logs
retention.purge	Weekly	Selfie and location retention, notification TTL
backup.verify	Daily	Restore test automation to a scratch instance
reminders	Daily	Birthdays, anniversaries, document expiry, declaration deadlines

16. Data Center Deployment (Self-Hosted)

16.1 Topology

Internet --> Edge firewall + WAF --> Load balancer (2x Nginx/HAProxy with keepalived VIP)

DMZ / Web tier: app-01, app-02 (Next.js) worker-01 (BullMQ) realtime-01 (optional)
| (private VLAN, firewall rules only)

Data tier: mongo-01 (PRIMARY) mongo-02 (SECONDARY) mongo-03 (SECONDARY or arbiter-free third member)
redis-01 (+ replica / Sentinel) minio-01..04 (erasure coded) or NAS with replication

Management: bastion + VPN, monitoring (Prometheus/Grafana/Loki), CI runners, Vault

Backup: backup-01 (PBM storage) --> encrypted copy to offsite site / second DC

16.2 Starting-point sizing (about 500 employees; adjust after load testing)

Component	Count	Spec (starting point)	Notes
App servers (Next.js)	2	4 vCPU, 8 GB RAM	Behind load balancer; stateless
Worker	1-2	4 vCPU, 8 GB RAM	Chromium for PDFs needs memory
MongoDB members	3	4-8 vCPU, 16-32 GB RAM, NVMe SSD	Different hosts/racks; RAM fits working set + indexes
Redis	1 + replica	2 vCPU, 4 GB RAM	Persistence on; Sentinel for failover
Object storage	1-4	4 vCPU, 8 GB, 2+ TB	Erasure coding if multi-node
Monitoring + CI + Vault	1-2	4 vCPU, 16 GB RAM	Separate from production data tier
Backup server	1	Large disk (3-5x data size)	Immutable or write-once storage if possible
Network	Dual links	Redundant switches, 10 GbE between data nodes	VLANs per tier
Power / cooling	-	UPS + generator, monitored temperature	Physical access control and logs

16.3 MongoDB operations

- **Replica set of 3 data-bearing members** across separate hosts (ideally separate racks or sites); write concern majority, read preference primary for transactional reads, secondaries only for reports and analytics.
- Enable authentication, keyfile or x.509 for member auth, TLS between members and clients, bind to private interface only.
- Separate DB users for app, worker, backup, monitoring and read-only analytics with least privilege.
- WiredTiger cache sizing, XFS filesystem, disable transparent huge pages, tune ulimits and swappiness per MongoDB production notes.
- Monitor replication lag, oplog window (keep 24-72 hours), cache usage, slow queries, connections, disk space; alert early.
- Plan upgrades one member at a time (secondaries first, step down primary last); test on staging; keep a rollback plan.
- Consider sharding only if data and throughput outgrow a replica set; not needed at HR scale.

16.4 Backup and disaster recovery

Item	Policy
Strategy	3-2-1 rule: 3 copies, 2 media types, 1 offsite (encrypted, preferably immutable)
MongoDB	Percona Backup for MongoDB: scheduled physical/logical full backups + continuous oplog slicing for point-in-time restore
Files	MinIO replication/versioning + periodic snapshot to backup storage
Config and code	Git repositories, infrastructure as code, Vault snapshots (secured)
Targets	RPO 15 minutes or better, RTO 2-4 hours (set with management; costs rise as targets tighten)

Item	Policy
Verification	Automated restore test weekly into scratch environment; quarterly full disaster drill with timing
Encryption keys	Back up the encryption keys separately from the data; losing keys means losing data
Runbooks	Documented steps for failover, restore, key rotation, security incident, certificate renewal

16.5 Environments and release

- **Dev** (local Docker Compose), **Staging** (mirrors production topology, anonymized data), **Production**.
- Blue-green or rolling deploys with health checks and automatic rollback; database migrations are versioned and backwards compatible (expand, migrate, contract).
- Feature flags to release safely; maintenance mode page; zero-downtime for web nodes.

16.6 Monitoring and alerting

- Metrics: request rate/latency/errors, CPU/RAM/disk, Mongo lag and slow queries, queue depth, failed jobs, login failures.
- Logs: structured JSON centralized in Loki with request IDs; audit logs separate and tamper-resistant.
- Uptime checks from outside the data center; on-call rotation and escalation; status page for internal users.

16.7 Mobile and push considerations

The mobile app connects to your public API endpoint over TLS, so the data center needs a public IP, valid certificates, WAF and DDoS protection. Push notifications require FCM/APNs; send only minimal, non-sensitive payloads and fetch details from your server after the app opens.

17. Quality, Testing and DevOps

Level	Tooling	What to cover
Unit	Vitest	Policy engines, accrual, payroll formulas, geofence logic, permission checks (aim for high coverage on core)
Integration	Vitest + MongoDB memory/replica test container	Services with real DB, transactions, workflows
API contract	OpenAPI + schema tests	Auth, validation, error shapes, permission matrix tests per role
End-to-end	Playwright	Login+MFA, clock in (mock geolocation), apply and approve leave, run payroll, download payslip
Load	k6	9 AM punch spike, payroll calculation of full headcount, report exports
Security	Dependency audit, SAST, container scan, annual pen test	OWASP Top 10, IDOR, privilege escalation, file upload
Accessibility	axe, manual keyboard and screen-reader checks	Critical flows
UAT	Real HR and accounts users in staging	Edge cases: mid-month joiners, notice buyout, arrears

17.1 CI/CD pipeline

```
commit -> lint + typecheck -> unit tests -> integration tests -> build -> container scan  
      -> deploy to staging -> e2e tests -> manual approval -> deploy to production (rolling) -> smoke tests
```

17.2 Definition of done (every feature)

- Permission checks implemented and tested for each role; tenant isolation tested.
- Validation and error states handled in UI and API.
- Audit log entries for sensitive actions.
- Loading, empty, error states designed; mobile layout checked; accessibility checked.
- Metrics and logs added; docs and OpenAPI updated; migration and rollback tested.

18. Compliance (India) and Data Governance

- **DPDP Act 2023:** notice and consent, purpose limitation, data minimization, correction and erasure rights, breach notification, retention limits, data principal grievance process.
- **Payroll statutory:** PF/EPF, ESI, Professional Tax (state-wise), Labour Welfare Fund, TDS under Section 192, gratuity, Bonus Act, Form 16 and 24Q data; keep all rates and ceilings configurable.
- **Labour and establishment laws:** leave entitlements, working hours, overtime and holiday rules vary by state and sector.
- **Records:** retention schedules for payroll and HR records; secure disposal at end of retention.
- **Access governance:** periodic role reviews, segregation of duties (preparer vs approver of payroll), audit logs reviewed.
- **Standards:** align processes with ISO 27001 / SOC 2 practices if you plan to offer the platform to other companies.

This section is planning guidance, not legal or tax advice. Have a Chartered Accountant and legal counsel validate statutory calculations, the privacy notice and the location-tracking policy before go-live.

19. Delivery Roadmap

Phase	Duration (indicative)	Deliverables	Exit criteria
0. Discovery and design	3-4 weeks	Requirements workshops with HR/Finance, user flows, Figma design system, ERD and collection design, infrastructure plan, threat model	Signed-off scope, designs and architecture
1. Foundation	6-8 weeks	Monorepo, CI/CD, infra baseline, auth + MFA, RBAC, org setup, employee master, audit log, notifications, file storage	All roles can log in; employee CRUD; audit trail; staging live
2. Attendance and leave	8-10 weeks	Mobile app, geofenced punch, policies, shifts, regularization, leave policies and ledger, workflow engine, holidays, calendars, live board	Pilot department uses it daily
3. Payroll and expenses	10-12 weeks	Salary structures, rules layer, payroll runs, payslips, tax and declarations, expenses, bank file, statutory reports	Parallel payroll matches 2-3 cycles
4. Hardening	4 weeks	Load tests, pen test, backup/restore drills, performance tuning, UAT fixes, documentation and training	Go-live checklist signed
5. Talent and growth	8-12 weeks	Recruitment, onboarding/offboarding, performance, assets, helpdesk, engagement, analytics	Replaces remaining spreadsheets
6. Advanced	Ongoing	Biometric integrations, AI features, multi-company, integrations (Slack/Teams/Tally), API marketplace	Per product roadmap

Pilot with one department for a month before company-wide rollout. Keep the old process running in parallel during the first payroll cycles.

19.1 Suggested team

- Product owner / business analyst with HR knowledge
- 2-3 full-stack TypeScript engineers, 1 mobile engineer, 1 UI/UX designer
- 1 DevOps/SRE and 1 DBA-capable engineer (essential for self-hosting)
- 1 QA engineer
- Part-time CA / HR-compliance advisor

20. Advanced Features Backlog

Area	Ideas
AI assistance	HR chatbot grounded on policies and the employee's own data (with permission checks); resume screening and ranking; attrition-risk prediction; anomaly detection in attendance and expenses; natural-language report builder; smart leave/approval suggestions
Attendance	Face recognition kiosk, geofence auto-reminders (notify when entering office), Bluetooth beacons, client-site visit tracking for field staff, shift-swap marketplace
Payroll	Multi-entity and multi-state, loans and advances, earned wage access, salary benchmarking, automated challan filing, accounting integration (Tally, Zoho Books)
Talent	Career pages, skill matrix, learning paths, internal job board, succession planning
Engagement	Pulse surveys, kudos with rewards, wellbeing resources, social feed
Platform	Public API and webhooks, custom report builder, form builder and custom fields, white-labeling, multi-company, localization for other countries, SSO/SCIM, event-sourced analytics warehouse
Integrations	Slack/Teams approvals, calendar sync, email-to-ticket, Aadhaar/DigiLocker verification, background check providers

21. Risks and Rules for AI Assistants

21.1 Top risks and mitigations

Risk	Mitigation
Payroll compliance errors	Configurable rules layer, CA review, parallel runs, variance reports, two-person approval
Geofence frustration or fraud	Start with soft policy and tune radius using real data; native app, attestation, device binding; transparent policy
Underestimated self-hosting effort	Dedicated DevOps, runbooks, automated backups and restore tests, monitoring from day one
Scope creep	Strict phase gates; ship attendance, leave and core HR before advanced modules
Data breach	Encryption, least privilege, audit logs, pen tests, incident plan, minimal data collection
MongoDB misuse (no transactions, missing indexes)	Replica set always, transactions for money and balances, index review, schema validation
Tenant data leakage	Tenant-scoped repositories, automated isolation tests, companyId in every index

21.2 Instructions for AI assistants using this document

- 1 Follow the structure in sections 4 and 6: thin route handlers, logic in services, data access in tenant-scoped repositories.
- 2 Always implement authorization in the service layer using section 2; always add companyId to queries.
- 3 Use TypeScript strict mode, Zod schemas shared between client and server, and typed errors.
- 4 Use MongoDB transactions for leave approval, payroll finalization and payouts; use append-only collections for punches, ledgers and audit logs.
- 5 Never trust client-reported location or time; follow the pipeline in section 9.3 and the query in section 9.4 (GeoJSON order [longitude, latitude]).
- 6 Never hardcode statutory rates, leave rules or policy values; read them from versioned configuration.
- 7 Build one module at a time in roadmap order; for each, deliver: schema, indexes, service, API, UI, tests, audit hooks, docs.
- 8 When a requirement is ambiguous, list assumptions explicitly and prefer the configurable option.
- 9 Do not introduce new infrastructure (message brokers, new databases) without an architecture decision record explaining why.

21.3 Open decisions to confirm before coding

Decision	Options	Recommendation
Mobile approach	React Native app vs PWA	React Native for reliable geofencing and anti-spoofing; PWA for secondary features
Auth provider	Auth.js vs Keycloak	Auth.js for simplicity; Keycloak if SSO/SAML across systems is needed
Object storage	MinIO vs Ceph vs NAS + S3 gateway	Choose after license and operations review
Orchestration	Docker Compose vs k3s/Kubernetes	Compose for first release; k3s when you need rolling scale and self-healing
Payroll scope at launch	Single state/entity vs multi-state	Single entity first; keep rules versioned for expansion
Geofence strictness at launch	Strict vs soft	Soft for first month, then tighten based on data
Retention periods	Selfies, raw location, HR records	Define with legal counsel; implement as configuration

22. Glossary

Term	Meaning
HRMS / HRIS	Human Resource Management System: software for managing employee data and HR processes
RBAC	Role-Based Access Control; permissions granted through roles, here with record scopes
Geofence	Virtual boundary (circle or polygon) around a physical location
GeoJSON / 2dsphere	Standard geographic data format and the MongoDB index type for earth-surface queries
Regularization	A request to correct or complete an attendance record
OD / WFH	On duty (working outside office) / work from home
LOP	Loss of pay: unpaid absence deducted from salary
F and F	Full and final settlement at employee exit
CTC	Cost to company: total annual employment cost
PF / ESI / PT / TDS	Provident Fund / Employees' State Insurance / Professional Tax / Tax Deducted at Source
Idempotency	Repeating the same request has the same effect as sending it once
RPO / RTO	Maximum acceptable data loss / maximum acceptable recovery time
PBM	Percona Backup for MongoDB
Modular monolith	A single deployable application with strongly separated internal modules
Effective dating	Storing changes with a start date so history is preserved and future changes can be scheduled